Introduction to pytest

What is pytest?

- A popular Python framework for automated testing.
- Verifies that functions and application logic produce expected results.
- Detects bugs early during development.
- Integrates with CI/CD pipelines, including Azure DevOps.

Why use pytest?

- Reduce manual testing.
- Catch regressions when code changes.
- Produce repeatable test results.
- Improve code reliability before deployment.

CasinoArizona training example: Verify that an API-processing function correctly handles operational records.

Installing pytest and Writing Your First Test

Installation

```
pip install pytest
```

Application code — calculator.py

```
def add(a, b):
    return a + b
```

Test code — test_calculator.py

```
from calculator
import add
def test_add():
    assert add(10, 20)
== 30
```

Execute the test

```
pytest -v
```

Key concepts

- Test functions should start with test_.
- assert compares actual and expected results.
- pytest discovers test files and functions automatically.

Expected result: 1 test passed.

Assertions, Test Cases and Failures

Assertions validate expected behavior.

```
def is_eligible(age):
    return age >= 18
```

Test multiple scenarios

```
from calculator import is_eligible
def test_eligible_adult():
    assert is_eligible(25) is True
def test_ineligible_minor():
    assert is_eligible(16) is False
def test_boundary_age():
    assert is_eligible(18) is True
```

Note: Place is_eligible() in calculator.py alongside the add() function, or adjust the import accordingly.

Important testing concepts

- Positive test: Expected valid input.
- Negative test: Expected rejection or failure behavior.
- Boundary test: Input at a limit, such as age 18.
- Failed assertion: Indicates actual behavior differs from expected behavior.

Discussion question: What happens if the function incorrectly uses age > 18 instead of age >= 18?

Testing API-Processing Logic

Scenario: Validate records retrieved from an API before exporting them.

Application code — processor.py

```
def process_posts(posts):
    return [
        {
            "id": post["id"],
            "title": post["title"].strip()
        }
        for post in posts
        if isinstance(post.get("id"), int)
        and isinstance(post.get("title"), str)
        and post["title"].strip()
    ]
```

Test code — test_processor.py

```
from processor import process_posts
def test_process_valid_post():
    posts = [{"id": 1, "title": " Daily Report "}]
    result = process_posts(posts)
    assert result == [
        {"id": 1, "title": "Daily Report"}
    ]
def test_empty_input():
    assert process_posts([]) == []
```

Run tests

```
pytest -v
```

Key takeaway: Test data-processing logic independently instead of making a live API request for every test.

pytest in Azure DevOps CI/CD

Automated testing workflow

1. Developer commits Python code to Azure Repos.
2. Azure Pipeline installs Python dependencies.
3. pytest runs automatically.
4. Test results determine whether the pipeline continues.
5. Successful validation allows the workflow to proceed to artifact publication and approved deployment stages.

Example pipeline commands

```
steps:
- script: |
    python -m pip
install pytest
    python -m
pytest -v
  displayName: Run
Python unit tests
```

Benefits

- Repeatable automated validation.
- Early detection of defects.
- Test execution on every relevant pipeline run.
- Greater confidence before release.

Trainer takeaway: A successful test run demonstrates that the tested cases passed; it does not prove the entire application is defect-free.